INFORME FINAL

Testing & Quality Assurance · Hito 3

Caja negra · Caja blanca · Gherkin

Daniel Bernal

https://github.com/CloudsCloudsClouds/examen_tqa_h3

2026-10-20

Aplicación bajo prueba: SauceDemo (Swag Labs)
https://www.saucedemo.com/

Presentación

Este informe consolida el trabajo de diseño de pruebas realizado para el Hito 3 de TQA sobre SauceDemo (Swag Labs). Incluye los 5 casos de caja negra, el análisis de caja blanca con 3 casos y 2 escenarios Gherkin. El alcance de este documento es el diseño de las pruebas; la ejecución y los reportes de bugs se documentan por separado.

ESTUDIANTE	Daniel Bernal	**MODALIDAD**	Individual
REPOSITORIO	https://github.com/CloudsCloudsClouds/examen_tqa_h3	**APLICACIÓN**	SauceDemo
ENTREGA	Hito 3 · TQA	**FECHA**	2026-10-20

Actividades cubiertas

Actividad	Contenido	Cantidad
Caja negra	Casos funcionales sobre SauceDemo	5
Caja blanca	Análisis de cobertura y casos sobre validar_compra	3
Gherkin	Escenarios BDD de login	2

Resumen del trabajo

La caja negra cubre inicio de sesión, validación de credenciales, flujo de carrito y checkout, compra de inicio a fin y ordenamiento del catálogo. La caja blanca analiza decisiones, condiciones y sentencias de `validar_compra`, elige cobertura de decisiones y explica el ajuste necesario para completarla. Gherkin documenta el login exitoso y la validación de credenciales incompletas o incorrectas.

Caja negra

Cinco casos funcionales diseñados desde la perspectiva del usuario sobre SauceDemo. Cada caso incluye objetivo, precondiciones, datos, pasos, resultado esperado y criterio de éxito.

ID	Título	Módulo	Tipo	Prioridad
TCCJ-001	Inicio de sesión válido y acceso al catálogo	Inicio de sesión	Smoke / Unitario	Alta
TCCJ-002	Validación de campos en el inicio de sesión	Inicio de sesión	Sanity / Unitario	Media
TCCJ-003	Carrito y checkout conservan la misma cantidad de productos	Carrito y checkout	Integración / Sanidad	Baja
TCCJ-004	Compra de inicio a fin (camino feliz)	Checkout / Compra	Sistema / Aceptación	Alta
TCCJ-005	Ordenamiento del catálogo por precio	Catálogo / Ordenamiento	Sanidad / Regresión	Baja

TCCJ-001 · Inicio de sesión válido y acceso al catálogo

ID DEL TC	TCCJ-001	**PRIORIDAD**	Alta
TÍTULO	Inicio de sesión válido y acceso al catálogo	**TIPO**	Smoke / Unitario
MÓDULO	Inicio de sesión	**AMBIENTE**	QA

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Confirmar que un usuario estándar puede iniciar sesión y llegar a la página de productos.

2. Precondiciones

Estado inicial	Usuario registrado y catálogo con productos disponibles.
Usuario / permisos	standard_user con contraseña secret_sauce.
Configuración	Navegador con acceso a https://www.saucedemo.com/.

3. Datos de prueba

Dato	Valor	Origen	Observación
URL	https://www.saucedemo.com/	SauceDemo	—
Usuario	standard_user	SauceDemo	—
Contraseña	secret_sauce	SauceDemo	—

4. Pasos y resultados esperados

#	Paso / acción	Dato utilizado	Resultado esperado
1	Abrir la página de inicio de sesión	https://www.saucedemo.com/	Se muestra el formulario de login.
2	Ingresar el usuario y la contraseña	standard_user / secret_sauce	Los campos quedan completos.
3	Presionar LOGIN	—	Se accede a la página de productos (catálogo).

5. Resultado esperado final

CRITERIO DE ÉXITO	El usuario standard_user inicia sesión y llega al catálogo. Si este caso falla, los demás casos de caja negra quedan invalidados.

TCCJ-002 · Validación de campos en el inicio de sesión

ID DEL TC	TCCJ-002	**PRIORIDAD**	Media
TÍTULO	Validación de campos en el inicio de sesión	**TIPO**	Sanity / Unitario
MÓDULO	Inicio de sesión	**AMBIENTE**	QA

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Comprobar que el formulario rechaza credenciales incompletas o incorrectas y mantiene al usuario en el inicio de sesión.

2. Precondiciones

Estado inicial	Ninguna.
Usuario / permisos	N/A
Configuración	Navegador con acceso a https://www.saucedemo.com/.

3. Datos de prueba

Dato	Valor	Origen	Observación
Usuario válido	standard_user	SauceDemo	—
Contraseña válida	secret_sauce	SauceDemo	—
Contraseña incorrecta	wrong_password	Preparado	—
Campos vacíos	Sin usuario o sin contraseña	Preparado	—

4. Pasos y resultados esperados

#	Paso / acción	Dato utilizado	Resultado esperado
1	Abrir el login, dejar el usuario vacío, ingresar la contraseña secret_sauce y presionar LOGIN	Usuario vacío / secret_sauce	La validación falla, se muestra el error del campo y el usuario permanece en el inicio de sesión.
2	Ingresar standard_user, dejar la contraseña vacía y presionar LOGIN	standard_user / contraseña vacía	La validación falla, se muestra el error del campo y el usuario permanece en el inicio de sesión.
3	Ingresar standard_user y wrong_password, presionar LOGIN	standard_user / wrong_password	La validación falla con un mensaje específico de usuario o contraseña incorrectos; el usuario permanece en el inicio de sesión.

5. Resultado esperado final

CRITERIO DE ÉXITO	Los tres escenarios de validación fallan de forma controlada, muestran el error correspondiente y no permiten el acceso.

TCCJ-003 · Carrito y checkout conservan la misma cantidad de productos

ID DEL TC	TCCJ-003	**PRIORIDAD**	Baja
TÍTULO	Carrito y checkout conservan la misma cantidad de productos	**TIPO**	Integración / Sanidad
MÓDULO	Carrito y checkout	**AMBIENTE**	QA

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Verificar que los productos y sus precios fluyen correctamente desde el catálogo hasta el resumen del checkout.

2. Precondiciones

Estado inicial	Usuario con credenciales válidas y catálogo disponible.
Usuario / permisos	standard_user
Configuración	Navegador con acceso a https://www.saucedemo.com/.

3. Datos de prueba

Dato	Valor	Origen	Observación
Producto 1	Sauce Labs Backpack ($29.99)	SauceDemo	—
Producto 2	Sauce Labs Bike Light ($9.99)	SauceDemo	—
Datos de checkout	Nombre: Test; Apellido: User; Zip: 12345	Preparado	—

4. Pasos y resultados esperados

#	Paso / acción	Dato utilizado	Resultado esperado
1	Iniciar sesión con standard_user	standard_user / secret_sauce	Se muestra la página de productos.
2	Agregar Sauce Labs Backpack y Sauce Labs Bike Light al carrito	Backpack $29.99 / Bike Light $9.99	El contador del carrito muestra 2 y ambos botones cambian a Remove.
3	Abrir el carrito	Carrito con 2 productos	Aparecen ambos productos con sus precios correctos y cantidad 1.
4	Presionar Checkout, ingresar nombre, apellido y código postal, y presionar Continue	Test / User / 12345	El resumen lista ambos productos, el Item total es $39.98 y el Total equivale al Item total más los impuestos.

5. Resultado esperado final

CRITERIO DE ÉXITO	Los productos agregados aparecen con precios correctos, el contador del carrito coincide con la cantidad de productos, el Item total del checkout es la suma de los precios y el Total es el Item total más los impuestos.

TCCJ-004 · Compra de inicio a fin (camino feliz)

ID DEL TC	TCCJ-004	**PRIORIDAD**	Alta
TÍTULO	Compra de inicio a fin (camino feliz)	**TIPO**	Sistema / Aceptación
MÓDULO	Checkout / Compra	**AMBIENTE**	QA

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Validar una compra completa desde el inicio de sesión hasta la confirmación y el regreso al catálogo.

2. Precondiciones

Estado inicial	Usuario con credenciales válidas y catálogo disponible.
Usuario / permisos	standard_user
Configuración	Navegador con acceso a https://www.saucedemo.com/.

3. Datos de prueba

Dato	Valor	Origen	Observación
Producto	Sauce Labs Bolt T-Shirt	SauceDemo	—
Datos de checkout	Nombre: Test; Apellido: User; Zip: 12345	Preparado	—

4. Pasos y resultados esperados

#	Paso / acción	Dato utilizado	Resultado esperado
1	Iniciar sesión con standard_user	standard_user / secret_sauce	Se muestra la página de productos.
2	Agregar Sauce Labs Bolt T-Shirt al carrito	Bolt T-Shirt	El contador del carrito muestra 1.
3	Abrir el carrito y presionar Checkout	Carrito con 1 producto	Se muestra Checkout: Your Information.
4	Ingresar nombre, apellido y código postal, y presionar Continue	Test / User / 12345	Se muestra Checkout: Overview.
5	Presionar Finish	—	Se muestra Thank you for your order!
6	Presionar Back Home	—	Se vuelve a la página de productos con el contador del carrito vacío.

5. Resultado esperado final

CRITERIO DE ÉXITO	El usuario completa la compra, ve la confirmación y vuelve al catálogo con el carrito vacío.

TCCJ-005 · Ordenamiento del catálogo por precio

ID DEL TC	TCCJ-005	**PRIORIDAD**	Baja
TÍTULO	Ordenamiento del catálogo por precio	**TIPO**	Sanidad / Regresión
MÓDULO	Catálogo / Ordenamiento	**AMBIENTE**	QA

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Comprobar que las opciones de ordenamiento reordenan correctamente la lista completa de productos.

2. Precondiciones

Estado inicial	Usuario con credenciales válidas y catálogo disponible.
Usuario / permisos	standard_user
Configuración	Navegador con acceso a https://www.saucedemo.com/.

3. Datos de prueba

Dato	Valor	Origen	Observación
Opciones de orden	Name (A to Z), Price (low to high), Price (high to low), Name (Z to A)	SauceDemo	—
Producto de referencia	Sauce Labs Backpack	SauceDemo	—

4. Pasos y resultados esperados

#	Paso / acción	Dato utilizado	Resultado esperado
1	Iniciar sesión con standard_user	standard_user / secret_sauce	El orden por defecto es Name (A to Z) y Sauce Labs Backpack aparece primero.
2	Seleccionar Price (low to high)	Price (low to high)	El orden es $7.99 Onesie, $9.99 Bike Light, $15.99 Bolt T-Shirt y Test.allTheThings() T-Shirt (Red) en cualquier orden, $29.99 Backpack, $49.99 Fleece Jacket.
3	Seleccionar Price (high to low)	Price (high to low)	El orden es exactamente el inverso del paso 2, con Fleece Jacket primero.
4	Seleccionar Name (Z to A)	Name (Z to A)	Test.allTheThings() T-Shirt (Red) aparece primero y Sauce Labs Backpack último.

5. Resultado esperado final

CRITERIO DE ÉXITO	Cada opción de ordenamiento reordena la lista completa de productos según su criterio y el orden por defecto es Name (A to Z).

Caja blanca

Código analizado:

```
def validar_compra(edad, cuenta_activa, saldo, monto):
    if edad < 18:
        return "Compra no permitida: edad"
    if not cuenta_activa:
        return "Compra no permitida: cuenta"
    if saldo >= monto and monto > 0:
        return "Compra aprobada"
    return "Compra rechazada"
```

Decisiones, condiciones y sentencias

ID	Expresión	Qué controla
D1	edad < 18	Rechaza la compra si el usuario es menor de edad.
D2	not cuenta_activa	Rechaza la compra si la cuenta no está activa.
D3	saldo >= monto and monto > 0	Aprueba la compra si hay saldo suficiente y el monto es positivo.

Condiciones simples:

ID	Expresión	Decisión
C1	edad < 18	D1
C2	not cuenta_activa	D2
C3	saldo >= monto	D3
C4	monto > 0	D3

Sentencias:

ID	Sentencia
S1	if edad < 18
S2	return «Compra no permitida: edad»
S3	if not cuenta_activa
S4	return «Compra no permitida: cuenta»
S5	if saldo >= monto and monto > 0
S6	return «Compra aprobada»
S7	return «Compra rechazada»

Criterio y mínimo de casos

Criterio: Cobertura de decisiones.

Se elige cobertura de decisiones porque cada `if` debe tomar sus dos resultados. Es más fuerte que cobertura de sentencias y más económica que cobertura de condiciones para el alcance de 3 casos. La cobertura de condiciones se analiza abajo, pero exigiría más casos por el cortocircuito de `and`.

Para cobertura de decisiones se necesitan al menos 4 casos. D1 debe ser verdadero y falso. D2 solo se evalúa cuando D1 es falso, así que necesita un caso con D1 falso y D2 verdadero, y otro con D1 falso y D2 falso. D3 solo se evalúa cuando D1 y D2 son falsos, así que necesita un caso con D3 verdadero y otro con D3 falso. Los caminos mínimos son:

1. D1 verdadero.
2. D1 falso, D2 verdadero.
3. D1 falso, D2 falso, D3 verdadero.
4. D1 falso, D2 falso, D3 falso.

Criterio	Mínimo	¿3 casos?	Observación
Sentencias	4	No	Falta ejecutar S7 (Compra rechazada).
Decisiones	4	No	Falta D3 falso.
Condiciones	5	No	Faltan C3 falso y C4 falso; por cortocircuito requieren casos separados.

Cobertura de los 3 casos

Caso	D1	D2	D3	Cobertura
TCCB-001	V	—	—	D1 verdadero; S2
TCCB-002	F	V	—	D1 falso, D2 verdadero; S4
TCCB-003	F	F	V	D1 falso, D2 falso, D3 verdadero; S6

Los 3 casos cubren D1 verdadero/falso, D2 verdadero/falso y D3 verdadero: 5 de los 6 resultados de decisión. Falta D3 falso.

Ajuste necesario

Para completar cobertura de decisiones falta un cuarto caso con D3 falso. Por ejemplo: `validar_compra(18, True, 30, 50)` → «Compra rechazada» (C3 falso, S7). Con ese caso, el mínimo de 4 se cumple.

Nota sobre tipos inválidos

La función no valida tipos. Si se pasan cadenas (`"ab"`, `"true"`, `"cien"`, `"A lot"`), Python lanza `TypeError` antes de evaluar las condiciones; no devuelve un mensaje de dominio. Ese comportamiento queda fuera del contrato de la función y no cuenta como cobertura de decisiones o condiciones. Si se quisiera documentar robustez, sería un caso adicional de tipo negativo por excepción, no uno de los 3 casos de cobertura.

Casos de caja blanca

ID	Título	Prioridad
TCCB-001	Edad menor de 18	Alta
TCCB-002	Cuenta inactiva	Media
TCCB-003	Compra aprobada	Alta

TCCB-001 · Edad menor de 18

ID DEL TC	TCCB-001	**PRIORIDAD**	Alta
TÍTULO	Edad menor de 18	**TIPO**	Caja blanca / Decisión
MÓDULO	validar_compra	**AMBIENTE**	Local / Python 3

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Verificar que la función rechaza la compra cuando la edad es menor de 18.

2. Precondiciones

Estado inicial	Función validar_compra definida y disponible.
Usuario / permisos	N/A
Configuración	Intérprete de Python 3.

3. Datos de prueba

Dato	Valor	Origen	Observación
edad	17	Preparado	—
cuenta_activa	True	Preparado	—
saldo	100	Preparado	—
monto	50	Preparado	—

4. Pasos y resultados esperados

#	Paso / acción	Dato utilizado	Resultado esperado
1	Ejecutar validar_compra(17, True, 100, 50)	17, True, 100, 50	Retorna «Compra no permitida: edad».

5. Resultado esperado final

CRITERIO DE ÉXITO	La función retorna «Compra no permitida: edad» y no evalúa las condiciones de cuenta ni de saldo.

6. Cobertura

DECISIÓN / CONDICIÓN	D1 verdadero (C1 verdadero); sentencia S2. No cubre D2 ni D3.

TCCB-002 · Cuenta inactiva

ID DEL TC	TCCB-002	**PRIORIDAD**	Media
TÍTULO	Cuenta inactiva	**TIPO**	Caja blanca / Decisión
MÓDULO	validar_compra	**AMBIENTE**	Local / Python 3

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Verificar que la función rechaza la compra cuando la cuenta no está activa, aunque la edad sea válida.

2. Precondiciones

Estado inicial	Función validar_compra definida y disponible.
Usuario / permisos	N/A
Configuración	Intérprete de Python 3.

3. Datos de prueba

Dato	Valor	Origen	Observación
edad	18	Preparado	—
cuenta_activa	False	Preparado	—
saldo	100	Preparado	—
monto	50	Preparado	—

4. Pasos y resultados esperados

#	Paso / acción	Dato utilizado	Resultado esperado
1	Ejecutar validar_compra(18, False, 100, 50)	18, False, 100, 50	Retorna «Compra no permitida: cuenta».

5. Resultado esperado final

CRITERIO DE ÉXITO	La función retorna «Compra no permitida: cuenta».

6. Cobertura

DECISIÓN / CONDICIÓN	D1 falso; D2 verdadero (C2 verdadero); sentencia S4. D3 no se evalúa.

TCCB-003 · Compra aprobada

ID DEL TC	TCCB-003	**PRIORIDAD**	Alta
TÍTULO	Compra aprobada	**TIPO**	Caja blanca / Decisión
MÓDULO	validar_compra	**AMBIENTE**	Local / Python 3

1. Objetivo

¿QUÉ SE QUIERE VALIDAR?	Verificar que la compra se aprueba cuando la edad es válida, la cuenta está activa y el saldo cubre un monto positivo.

2. Precondiciones

Estado inicial	Función validar_compra definida y disponible.
Usuario / permisos	N/A
Configuración	Intérprete de Python 3.

3. Datos de prueba

Dato	Valor	Origen	Observación
edad	18	Preparado	—
cuenta_activa	True	Preparado	—
saldo	100	Preparado	—
monto	50	Preparado	—

4. Pasos y resultados esperados

#	Paso / acción	Dato utilizado	Resultado esperado
1	Ejecutar validar_compra(18, True, 100, 50)	18, True, 100, 50	Retorna «Compra aprobada».

5. Resultado esperado final

CRITERIO DE ÉXITO	La función retorna «Compra aprobada».

6. Cobertura

DECISIÓN / CONDICIÓN	D1 falso; D2 falso; D3 verdadero (C3 verdadero y C4 verdadero); sentencia S6.

Gherkin

Escenarios BDD para el inicio de sesión en SauceDemo. Se incluye un camino exitoso y una validación con credenciales incompletas o incorrectas.

```
Feature: Login

  @smoke
  Scenario: Valid login reaches Products page
    Given I am on the login page at https://www.saucedemo.com/
    When I log in with username "standard_user" and password "secret_sauce"
    Then I should see the Products page with six items

Feature: Login validation

  @unit
  Scenario Outline: Login rejects missing or incorrect credentials
    Given I am on the login page
    When I log in with username "<username>" and password "<password>"
    Then I should see the error "<error>"
    And I should still be on the login page

    Examples:
      | username      | password       | error
|
      |               | secret_sauce   | Epic sadface: Username is required
|
      | standard_user |                | Epic sadface: Password is required
|
      | standard_user | wrong_password | Epic sadface: Username and password do not match any user in this service
|
```

Conclusiones

- La caja negra cubre los flujos principales de SauceDemo con 5 casos funcionales distintos.
- La caja blanca elige cobertura de decisiones: los 3 casos cubren 5 de 6 resultados; falta `D3 falso` y se propone un cuarto caso.
- La cobertura de condiciones exigiría 5 casos por el cortocircuito de `and`.
- Gherkin documenta el login exitoso y la validación de credenciales en 2 escenarios.
- El repositorio con el trabajo completo está en https://github.com/CloudsCloudsClouds/examen_tqa_h3.